

BSc (Hons) in **Computer Science****SE3062 : Intelligent Systems**

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing**Practical 01**

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation**Task 0 : Setting up and getting the starter Code**

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P-performance measure,E-enviroment,A-actuator,s-sensor

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

```
self.width  
self.height  
self.walls  
self.food_positions  
self.opponents  
self.agent_pos
```

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

The environment explicitly initializes and manages "self.opponents" which represent independent adversarial entities moving around the grid, making decisions and interacting with the same shared space. Since there are multiple active entities (the main agent plus one or more opponents) operating within and affecting the environment state simultaneously, it constitutes a multi-agent system rather than a single-agent one.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

The absolute, cumulative historical log of every single percept the agent has received from its initialization up to the current moment.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensors are the mechanism by which an agent receives inputs or information about its current state and surroundings from the environment. Similarly, "get_percept()" gathers internal game states (like position, food detection, and score) and packages them into a dictionary to feed into the agent so it can perceive what is happening around it.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what

the agent can and cannot see.

agent receives exact information about food smells and wall hitting from sensors. but this agent receives sensor inputs only. cuz this is a partially observable agent.

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

In the PEAS framework, the "Performance Measure" evaluates how successful an agent's behavior is based on a set of criteria. Changing the `self.score` variable (subtracting points for hitting a wall or adding points for eating food) directly updates the numeric metric used to evaluate the agent's performance in the environment.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is structurally important that the performance metric evaluates external environment interactions such as hitting a wall or collecting food rather than internal computational effort because an AI agent's rationality is defined entirely by its success in achieving external goals within its world. The performance measure exists to judge the quality of the agent's behavior and decision-making relative to the task, maintaining a clear separation of concerns where software efficiency handles computational costs while the performance metric evaluates task success. Rewarding or penalizing internal processing effort would fundamentally misalign the incentive structure, encouraging shorter or simpler code rather than smarter, goal-oriented navigation and survival strategies designed to influence the external environment effectively.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Adding "self.toxic_traps" while hiding them from the agent's sensors keeps the environment Partially Observable. Since the agent cannot perceive where these hazards are located, it lacks complete information about the environment state, introducing hidden risks and uncertainty into its decision-making.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

In artificial intelligence, an agent is considered rational if it selects actions that maximize its expected performance measure or utility.

By perceiving the presence of a toxic trap, the agent gains the necessary feedback to alter its behavior such as choosing an alternative path or retreating thereby actively avoiding severe score penalties and danger. This sensory input replaces blind guessing with informed, goal-oriented decision-making, directly increasing its chances of survival and maximizing its overall utility.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

vacuum cleaner's performance based strictly on "the total amount of dirt cleaned up". a rational agent will strictly maximize its defined performance metric, a clever agent exploited this faulty metric by cleaning up the dirt, dumping it back out onto the floor, and cleaning it up repeatedly to artificially inflate its score without actually keeping the floor clean.

Finalize and Lock Lab 01 Analytical Evaluation

Lab 01 code analysis and theoretical evaluation complete and verified!

